UNIVERSIDADE DE BRASÍLIA - CAMPUS GAMA

FACULDADE DE CIÊNCIAS E TECNOLOGIAS EM ENGENHARIA

INSTITUTO DE PESQUISAS ELDORADO

TIC EM TRILHAS PARA INTELIGÊNCIA ARTIFICIAL: FORMAÇÃO EM IA

RESIDÊNCIA EM INTELIGÊNCIA ARTIFICIAL:

QUALIDADE DE VIDA: EMPODERANDO PESSOAS COM IA PARA COMPREENDER MELHOR, DECIDIR MELHOR E VIVER MELHOR

Assistente Virtual Tira Dúvidas - IA para facilitar o acesso a benefícios socioassistenciais.

GRUPO 7

Ivan Vieira Delmondez de Oliveira

Giulio dos Anjos Santos

Ian Lucca Soares Mesquita

Paulo Henrique Sousa Soares e Silva

Thayna Gonçalves Dutra

BRASÍLIA - DISTRITO FEDERAL

2026

Sumário

1. Introdução e Contextualização **3**
2. Artefatos do Sistema **3**
2.1 Base de Conhecimento Curada 3
2.2 Estratégia de Chunking e seu Impacto no Domínio Jurídico 3
2.3 Estratégia de Embeddings e a Escolha do Modelo Local 4
2.4 Implementação do Banco Vetorial 4
3. Arquitetura e Engenharia do Sistema **4**
3.1 Pipeline de Ingestão de Dados 4
3.2 Arquitetura de Execução do RAG 6
4. Demonstração e Validação (Rastreabilidade) **7**
4.1 Cenário de Teste Executado 7
4.2 Recuperação e Rastreabilidade 7
4.3 Resposta Fundamentada e Geração 7
5. Reprodutibilidade e Testes **7**
5.1 Comandos de Inicialização do Ambiente 7
5.2 Teste Unitário Automatizado 8
6. Mapeamento de Arquivos do Projeto **9**

1. Introdução e Contextualização

Este documento apresenta a especialização técnica e a validação do protótipo de **Geração Aumentada por Recuperação** (**RAG**) desenvolvido para o ecossistema de assistente virtual institucional destinado a apoiar cidadãos do Distrito Federal na identificação de potenciais benefícios sociais. O sistema realiza a ingestão de bases estruturadas e textos legislativos complexos, permitindo que perguntas formuladas em linguagem natural sejam respondidas com alta fidelidade factual.

O foco primário desta etapa reside na qualidade de recuperação da informação, na estrita fundamentação das respostas e na rastreabilidade total das fontes utilizadas, mitigando riscos de alucinação por parte do Modelo de Linguagem (LLM).

2. Artefatos do Sistema

2.1 Base de Conhecimento Curada

A base de conhecimento do protótipo foi consolidada a partir de duas frentes principais de dados oficiais, garantindo a abrangência necessária para o domínio de programas sociais:

Dados Estruturados: Um arquivo CSV contendo **42 registros** catalogados e mapeados sobre programas sociais federais e distritais.
Dados Não Estruturados (Legislação): Um ecossistema de **27 textos** integrais de legislações correlatas, coletados e extraídos diretamente de portais e páginas governamentais oficiais.
Volumetria Total do Índice: A base consolidada gerou um índice final com **1.204 trechos** (**chunks**) carregados com sucesso no banco de dados.

2.2 Estratégia de Chunking (Segmentação) e seu Impacto no Domínio Jurídico

A segmentação de dados (chunking) é uma das etapas mais críticas para o sucesso de um sistema RAG, pois o modelo de linguagem (LLM) só conseguirá responder corretamente se o contexto recuperado contiver a informação exata e sem cortes.

No nosso sistema, foi implementada uma estratégia híbrida adaptada à natureza dos dados coletados:

Segmentação Semântica para Dados Estruturados (CSV): O arquivo que foi nomeado como "programassociais_legislacao_links_oficiais" possui uma estrutura rígida. Se dividirmos o texto de forma puramente matemática, correríamos o risco de separar o nome do programa dos seus critérios de elegibilidade. Para evitar isso, cada registro do CSV é convertido em um documento derivado em markdown e quebrado estritamente em **4 seções semânticas** fixas e contextualizadas. Por exemplo: Descrição, Requisitos, Benefício e Canais de Atendimento. Isso garante que o embedding capture o bloco de informação completo de cada programa.
Segmentação Por Janela Deslizante para Textos Legislativos: Leis e decretos possuem uma estrutura longa e complexa. Para esses arquivos, adotamos um limite de **1.200 caracteres** por bloco (chunk size). Essa extensão é ideal para acomodar artigos, parágrafos e incisos internos dentro de um mesmo valor.
Mecanismo de Sobreposição de 180 Caracteres: A sobreposição funciona como uma "janela deslizante". Os últimos **180 caracteres** de um bloco são repetidos no início do bloco seguinte. Isso garante que a continuidade de um raciocínio jurídico ou uma regra de transição não seja perdida, independentemente de onde ocorra o corte do algoritmo.

Ajuste Fino por Espaçamento: O algoritmo de corte não interrompe o texto de forma arbitrária no caractere 1.200. Ele busca o espaço em branco ou fim de frase mais próximo antes desse limite. Isso impede que palavras ou termos técnicos de legislação sejam cortados pela metade, preservando a integridade linguística na hora de gerar o embedding.

2.3 Estratégia de Embeddings e a Escolha do Modelo

Os embeddings transformam os blocos de texto em vetores matemáticos (coordenadas em um espaço multidimensional) para que o sistema consiga calcular a proximidade semântica entre a dúvida do usuário e a base de conhecimento.

Nossa estratégia técnica baseia-se em:

Arquitetura de Embeddings Simétrica e Local: Foi utilizado o modelo bge-m3 orquestrado localmente através do framework Ollama. Adotar o exato mesmo modelo tanto na fase de ingestão quanto na fase de busca é um requisito obrigatório para garantir que as perguntas do usuário "leiam" o espaço vetorial na mesma escala matemática em que os documentos foram indexados.
Justificativa de Desempenho do bge-m3: O modelo bge-m3 BAAI General Embedding (BGE) foi escolhido por suas propriedades avançadas. Ele é um modelo nativamente multilingue com excelente acurácia na língua portuguesa e foi treinado para lidar de forma robusta com textos técnicos, governamentais e jurídicos. Ele possui uma capacidade superior de compreender sinônimos, como por exemplo: correlacionar "ajuda para o gás" com o termo formal "Auxílio Gás".
Descrição do Cenário Processamento Otimizado em Lotes (Batches) de 32: Durante o processo de ingestão ingestao.py, o envio dos trechos de texto para o modelo não é feito um a um, o que sobrecarregaria o hardware e tornaria o processo lento. O pipeline agrupa os blocos em lotes de até 32 trechos por vez. Isso maximiza o uso de memória de processamento local, acelera a criação dos vetores e mantém o sistema estável sem perda de pacotes de dados.

2.4 Implementação do Banco Vetorial

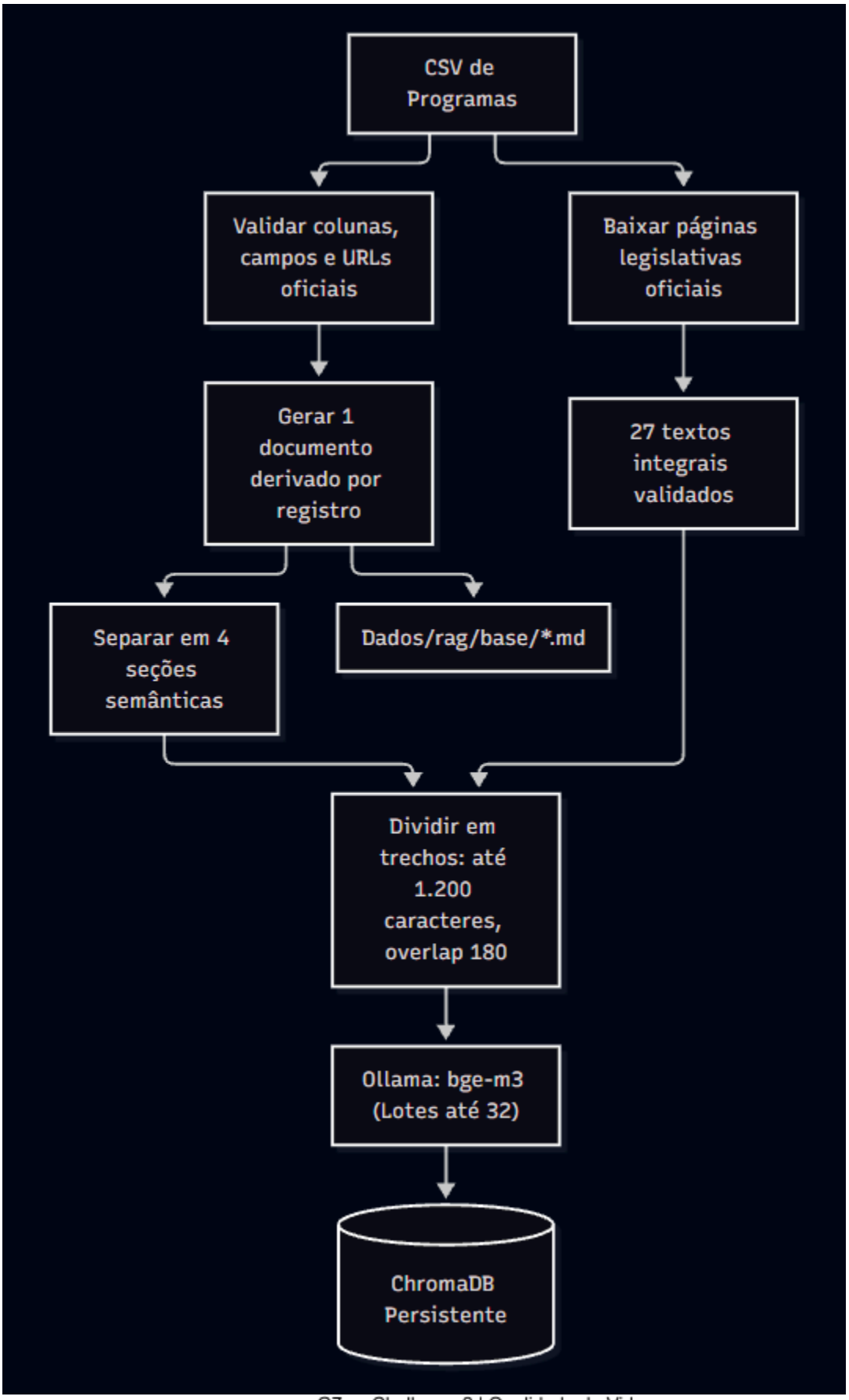
O armazenamento e indexação dos embeddings foi construído com foco em persistência e performance de busca local:

Tecnologia Utilizada: Chrome DB persistente.
Coleção Ativa: fontes_oficiais_df.
Métrica de Distância: Distância Cosseno para o cálculo de similaridade vetorial.
Infraestrutura Local: No ambiente Windows, o banco localiza-se na variável de ambiente %LOCALAPPDATA%\AssistenteVirtualDF\vetor-v2, sendo totalmente parametrizável via chave VECTOR_DB_PATH. Os arquivos **Hierarchical Navigable Small World** (**HNSW**) foram verificados e validados no diretório de destino.

3. Arquitetura e Engenharia do Sistema

3.1 Instituições e domínio de informação

O pipeline de ingestão é o fluxo responsável por extrair os dados brutos de suas fontes originais, limpá-los, fragmentá-los e convertê-los em valores para o banco de dados.

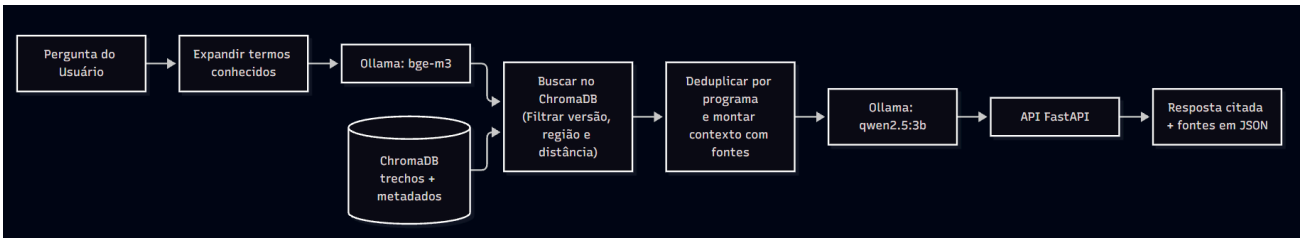

CSV de Programas
Validar colunas, campos e URLs oficiais
Baixar páginas legislativas oficiais
Gerar 1 documento derivado por registro
27 textos integrais validados
Separar em 4 seções semânticas
Dados/rag/base/*.md
Dividir em trechos: até 1.200 caracteres, overlap 180
Ollama: bge-m3 (Lotes até 32)
ChromaDB Persistente

Descrição Detalhada do Fluxo de Ingestão:

1. **Extração Bifásica de Dados:** O pipeline inicia-se em duas frentes. Primeiro na leitura do arquivo estruturado programas_sociais_legislacao_links_oficiais.csv. Simultaneamente, o script baixar_fontes.py varre a internet fazendo web scraping das URLs oficiais contidas no CSV, extraído o texto limpo de **27 textos legislativos integrais**.
2. **Validação de Integridade:** Antes de qualquer manipulação, o sistema valida a presença das colunas obrigatórias no CSV e testa a acessibilidade dos links. Se houver falha de conexão como a página fora do ar, o sistema emite um alerta, isolando o registro para não corromper o índice.
3. **Renderização e Segmentação Semântica:** Cada linha válida do CSV é transformada em um arquivo Markdown intermediário dentro da pasta BancodeDadoseRAG/rag/base/. Esse arquivo é fatiado cirurgicamente em **4 seções semânticas fixas**. Paralelamente, os textos das leis são consolidados para a etapa de corte.
4. **Processamento em Chunks:** Tanto as seções dos programas quanto as leis consolidadas passam pelo algoritmo de chunking (segmentação), gerando blocos textuais otimizados de até **1.200 caracteres com sobreposição de 180 caracteres**, respeitando o espaçamento das palavras.
5. **Vetorização Otimizada (Embedding):** Os 1.204 blocos textuais resultantes são agrupados em **lotes de até 32 unidades** e enviados ao modelo local **bge-m3** via Ollama. O modelo converte cada bloco de texto em uma representação matemática vetorial de alta dimensionalidade.
6. **Persistência Vetorial:** Os vetores gerados, juntamente com seus metadados essenciais como título do programa, órgão responsável e URL original, são gravados permanentemente no **ChromaDB**, consolidando o índice na coleção fontes_oficiais_df.

3.2 Arquitetura de Execução do RAG (Runtime Pipeline)

Esta seção descreve o ciclo de vida de uma requisição de usuário, desde o momento em que a pergunta é feita na API até o retorno da resposta fundamentada.



Descrição Detalhada do Ciclo de Execução (Query Time):

1. **Entrada da Pergunta e Expansão de Termos:** A pergunta enviada em linguagem natural pelo usuário (exemplo: “Tem ajuda para comprar o botijão de gás?”) é recebida pelo back-end desenvolvido em **FastAPI**. Antes de buscar no banco, o sistema aplica uma camada de **expansão de termos** conhecidos para alinhar sinônimos populares à linguagem formal jurídica.
2. **Vetorização da Consulta:** A pergunta expandida é submetida ao mesmo modelo **bge-m3** no Ollama. O modelo gera um vetor numérico que representa o significado semântico da dúvida do usuário.
3. **Busca Vetorial e Filtragem por Metadados (Retrieval):** O vetor da pergunta é comparado com os vetores armazenados no **ChromaDB**. O banco realiza o cálculo de **Distância Cosseno** para encontrar os trechos de texto mais próximos ou similares. Nesta etapa, o sistema aplica filtros rígidos de metadados para garantir que apenas trechos pertinentes à região e à versão atualizada do programa sejam considerados, descartando textos com baixa similaridade

.

4. **Deduplicação e Montagem do Contexto:** Os blocos de texto mais relevantes retornados pelo Chroma DB passam por um algoritmo de **deduplicação**. Se múltiplos trechos pertencerem ao mesmo programa ou lei, eles são estruturados de forma limpa para não desperdiçar a janela de contexto do modelo. O sistema monta um "prompt enriquecido", injetando esses trechos recuperados junto com suas respectivas fontes de origem.
5. **Geração Fundamentada** (**Generation**)**:** O prompt estruturado é enviado ao LLM **gwen2.5:3d** via Ollama. O modelo é instruído estritamente a responder à dúvida do usuário **utilizando apenas as informações contidas no contexto injetado**. Caso a resposta não esteja nos trechos, o modelo é treinado para reportar que não encontrou a informação, eliminando alucinações.
6. **Resposta Estruturada com Rastreabilidade:** O modelo gera o texto da resposta, e a API FastAPI formata o payload final em formato JSON. O resultado final devolvido ao usuário contém a resposta em texto claro acompanhada das **evidências das fontes** (**título, URL, oficial e o trecho exato utilizado**), provando a rastreabilidade necessária.

4. Demonstração e Validação (Rastreabilidade)

Conforme os critérios de avaliação definidos, o foco da demonstração é validar a robustez da recuperação semântica e a veracidade empírica do texto gerado perante as fontes primárias.

4.1. Cenário de Teste Executado

Consulta Submetida: "Tem ajuda para comprar o botijão de gás?"

4.2 Recuperação e Rastreabilidade

A execução do script de busca buscar.py validou com sucesso o isolamento e a pontuação dos blocos de dados. O motor de busca isolou com precisão os metadados dos programas de transferência de renda federais e distritais:

Chunks Recuperados: Segmentos diretos do **Auxílio Gás** e do **Bolsa Família**.
Evidências de Fontes Encontradas: O sistema vinculou os blocos de texto diretamente aos seus pilares normativos:
↪ **Lei n° 15.348/2026** - Legislação Distrital/Federal correlata.
↪ **Lei n° 14.601/2024** - Marcos Regulatório do Bolsa Família.
Metadados Retornados: Cada bloco recuperado expõe de forma transparente o nome do programa, o órgão público responsável, o score de similaridade cosseno, o trecho de texto exato e a **URL governamental oficial** de validação.

4.3 Resposta Fundamentada e Geração

A requisição enviada ao endpoint de produção POST/api/chat retornou sucesso **HTTP** 200. O modelo **gwen2.5:3b** utilizou exclusivamente o contexto injetado para formular a resposta, estruturando-a em formato JSON que contém as chaves explícitas de atribuição (**title**, **url** e o trecho de texto que originou a afirmação).

5. Reprodutibilidade e Testes

5.1 Comandos de Inicialização do Ambiente

Para replicar a infraestrutura e os testes validados nesta documentação técnica, é necessário executar os blocos de comando abaixo em ambiente Windows PowerShell (com o ambiente virtual ativado):

Passo 1: Download e inicialização dos modelos locais no Ollama
ollama pull bge-m3
ollama pull gwen2.5:3b

Passo 2: Execução dos scripts de coleta e ingestão na raiz do projeto
python .\BancoDeDadoseRAG\rag\baixar_fontes.py
python .\BancoDeDadoseRAG\ingestao.py

Passo 3: Inicialização do servidor Back-end via FastAPI
python -m uvicom main:app -reload -app-dir .\Back-end

5.2 Teste Unitário Automatizado

A estabilidade matemática e lógica das rotinas foi validada por meio de testes automatizados integrados. A cobertura atual inclui:

Testes do RAG e da Base BancoDeDadoseRAG/rag**: 20 testes aprovados**
Testes do Back-end Back-end**: 11 testes aprovados**
Acurácia do Pipeline de Recuperação: O script de avaliação avaliar.py aferiu uma taxa de acerto de **11/11** (**100%**) no posicionamento do programa esperado dentro do Top-5 resultados retornados pelo banco vetorial.

Passo 1: Execução da suíte completa de testes
python -m unittest discover -s ./Back-end -p “test_*.py”
python -m unittest discover -s ./BancoDeDadoseRAG/rag -p “test_*.py”

6. Mapeamento de Arquivos do Projeto

Os artefatos e códigos-fonte que compõem este entregável estão distribuídos na estrutura oficial do projeto conforme o mapeamento abaixo:

BancoDeDadoseRAG/rag/README.md**:** Documentação técnica de implementação, limitações conhecidas e dicionário de comandos.
BancoDeDadoseRAG/rag/baixar_fontes.py **e** BancoDeDadoseRAG/rag/fontes.py**:** Rotinas de web scraping, coleta e extração limpa de textos legislativos oficiais.
BancoDeDadoseRAG/rag/ingestao.py**:** Script responsável pelo gerenciamento de batches e carga dos embeddings no ChromaDB.
BancoDeDadoseRAG/rag/dataset.py**:** Mecanismo de preparação da base bruta e execução da estratégia de chunking.
BancoDeDadoseRAG/rag/rag.py**:** Arquitetura lógica do pipeline RAG (orquestração entre busca vetorial e prompting).
BancoDeDadoseRAG/rag/buscar.py**:** Utilitário de linha de comando para validação isolada da etapa de recuperação e scores.
BancoDeDadoseRAG/programas_sociais_legislacao_links_oficiais.csv: Base de dados de entrada curada contendo o mapeamento inicial dos programas.